



MKD

2026-06-21

1 Contest

2 Number theory

Contest (1)

dbg.h20 lines

```
#define dbg(...) cerr<<"DEBUG:["<<#__VA_ARGS__<<"] = [" ,\n" _print(__VA_ARGS__),cerr<<"\n"
template<class T, class=void>struct cntnr : false_type {};
template<class T>struct cntnr<T,
void_t<decltype(declval<T>()).begin()),
decltype(declval<T>().end()),
typename T::value_type> > : true_type {};
template<class T>using raw=remove_cv_t<remove_reference_t<T> >;
template<class T>enable_if_t<!cntnr<T>::value>
_print(const T &x) { cerr << x; }
template<class T>enable_if_t<cntnr<T>::value>_print(const T &v){
cerr << "["; bool f = false;
for (const auto &x: v)
f ? cerr << (cntnr<raw<decltype(x)> >::value ?",\n":"", "):
cerr, f = true, _print(x);
cerr << "];";
}
template<class T, class ... A>
void _print(const T &x, const A &... a) {
_print(x);(cerr << " , ", _print(a)), ...);};
```

template.cpp25 lines

```
#include <bits/stdc++.h>
using namespace std;
// #define LOCAL_DEBUG
#ifdef LOCAL_DEBUG
#include "dbg.h"
#else
#define dbg(...) 42
#endif
#define rep(i, a, b) for(int i = a; i < (b); ++i)
#define all(x) begin(x), end(x)
#define sz(x) (int)(x).size()
using ll = long long;
using pii = pair<int, int>;
using vi = vector<int>;
void init() {}
void solve() {}
signed main() {
init();
cin.tie(nullptr), ios::sync_with_stdio(false);
// cin.exceptions(cin.failbit);
int T = 1;
// cin >> T;
while (T--)solve();
return 0;
}
```

1.1 checker

cek.sh19 lines

```
#!/usr/bin/env bash
set -u
cd "$(dirname "$0")"
g++ gen.cpp -o gen -std=c++17 -O2
g++ std.cpp -o std -std=c++17 -O2
g++ sol.cpp -o sol -std=c++17 -O2
cnt=0
```

```
while true; do
cnt=$((cnt + 1))
echo "Testing case #$cnt"
./gen "${1-}" > in.txt
./std < in.txt > out1.txt
./sol < in.txt > out2.txt
if ! diff -q out1.txt out2.txt > /dev/null; then
echo "WA on case #$cnt"
diff out1.txt out2.txt
read -r -p "Press Enter to continue ..."
fi
done
```

cek.bat19 lines

```
@echo off
cd "%~dp0"
g++ gen.cpp -o gen -std=c++17 -O2
g++ std.cpp -o std -std=c++17 -O2
g++ sol.cpp -o sol -std=c++17 -O2
set cnt=0
:loop
set /a cnt+=1
echo Testing case %cnt%
gen %1 > in.txt
std < in.txt > out1.txt
sol < in.txt > out2.txt
fc out1.txt out2.txt > nul
if errorlevel 1 (
echo WA on case %cnt%
fc out1.txt out2.txt
pause
)
goto loop
```

gen.h22 lines

```
uint64_t seed = chrono::steady_clock::now()
.time_since_epoch().count() ^ random_device {} ();
mt19937 rng(seed);
template<class T>
T randi(T l,T r){return uniform_int_distribution<T>(l,r)(rng);}
vi randvi(int n, int l, int r) {
vi v(n);
rep(i, 0, sz(v))v[i] = randi(l, r);
return v;
}
vi randp(int n) {
vi p(n);
iota(all(p), 1);
shuffle(all(p), rng);
return p;
}
int main(int argc, char *argv[]) {
if (argc >= 2)
seed = stoull(argv[1]), rng.seed(seed);
cerr << seed << endl;
return 0;
}
```

Number theory (2)

2.1 模运算

ModularArithmetic.h2f9ed0, 18 lines

```
Description: Operators for modular arithmetic. You need to set mod to
some number first and then you can use the structure.
"euclid.h"
```

constexpr ll MOD = 17; // change to something else
struct Mod {
ll x;
Mod(ll _x) : x((_x % MOD + MOD) % MOD) {}
Mod operator+(Mod b) { return Mod(((x + b.x)%MOD)%MOD); }
Mod operator-(Mod b) { return Mod(((x - b.x)%MOD+MOD)%MOD); }
Mod operator*(Mod b) { return Mod((x * b.x %MOD+MOD)%MOD); }
Mod operator/(Mod b) { return *this * invert(b); }
Mod invert(Mod a) {
ll x, y, g = euclid(a.x, MOD, x, y);
assert(g == 1);return Mod((x % MOD + MOD) % MOD);
}
Mod operator^(ll e) {
if (!e) return Mod(1);
Mod r = *this ^ (e / 2); r = r * r;
return e & 1 ? *this * r : r;
}
};

模逆元

$$\frac{a}{b} \equiv a \cdot b_{\text{inv}} \equiv x \pmod{p}$$
$$\Rightarrow a \equiv bx \pmod{p}$$
$$\Rightarrow a \cdot b_{\text{inv}} \cdot b \equiv bx \pmod{p}$$
$$\Rightarrow a \cdot (b_{\text{inv}} \cdot b - 1) \equiv 0 \pmod{p}$$
$$\Rightarrow b_{\text{inv}} \cdot b \equiv 1 \pmod{p}$$
$$\text{inv存在} \Leftrightarrow \gcd(b, p) = 1$$
$$b \cdot b_{\text{inv}} = k \cdot p + 1, g = \gcd(b, p) \Rightarrow g \cdot b_1 \cdot b_{\text{inv}} = k \cdot g \cdot p_1 + 1$$
$$\Rightarrow g \cdot (b_1 \cdot b_{\text{inv}} - k \cdot p_1) = 1$$
$$\text{数域为}\mathbb{Z} \Rightarrow g = 1, b_1 \cdot b_{\text{inv}} - k \cdot p_1 = 1$$

ModInverse.h6f684f, 3 lines

```
Description: Pre-computation of modular inverses. Assumes LIM ≤ mod
and that mod is a prime.
constexpr ll mod = 1000000007, LIM = 200000;
ll* inv = new ll[LIM] - 1; inv[1] = 1;
rep(i,2,LIM) inv[i] = mod - (mod / i) * inv[mod % i] % mod;
```